



Day 04 — R Programming

Operators, writing your own functions (with default arguments) and anonymous functions, for-loops, nested loops, `expand_grid()`, and `rowwise()`

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-09-03

Topics:

Package management	2
Getting the data	2
Operators	3
Writing your own functions	5
Anonymous functions	8
For loops	9
Rowwise	18
Try it: who cleared the threshold?	19
Try it: counting who cleared	20
Try it: average share by pollster	21

Package management

Same as every session: bootstrap pacman, then load the tidyverse.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
```

Getting the data

Same wide polling data as Day 3 — one column per party.

```
polls <- read_csv("d04/polls.csv")
```

MESSAGE

Rows: 2077 Columns: 12

— Column specification —

Delimiter: ","

chr (3): Poll_ID, Pollster, Period

dbl (8): n, CDUCSU, SPD, Gruene, FDP, Linke, AfD, BSW

date (1): Date

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
party_cols <- c("CDUCSU", "SPD", "Gruene", "FDP", "Linke", "AfD", "BSW")
polls |> head(100)
```

OUTPUT

A tibble: 100 × 12

	Poll_ID	Pollster	Date	n	Period	CDUCSU	SPD	Gruene	FDP	Linke
	<chr>	<chr>	<date>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	BT021701040	Forsa	2017-01-04	2001	27.12...	37	21	10	6	9
2	BT01170105P	Infrates...	2017-01-05	1505	02.01...	37	20	9	5	9
3	BT03170107L	Verian	2017-01-07	834	03.01...	38	22	10	5	9
4	BT02170111P	Forsa	2017-01-11	2501	02.01...	37	20	10	6	9
5	BT03170114M	Verian	2017-01-14	1878	06.01...	37	21	10	6	10
6	BT02170118Q	Forsa	2017-01-18	2503	09.01...	38	21	9	6	9
7	BT03170121N	Verian	2017-01-21	2805	12.01...	36	21	9	6	11
8	BT02170125R	Forsa	2017-01-25	2504	16.01...	37	21	10	6	9
9	BT01170127Q	Infrates...	2017-01-27	1029	24.01...	35	23	9	6	8
10	BT03170128O	Verian	2017-01-28	1996	19.01...	37	23	10	6	10

i 90 more rows

i 2 more variables: AfD <dbl>, BSW <dbl>

Operators

Same combined filter as Day 3 (`bsw_forsa`), adapted to this year's wide table — no `party` column here, so “was BSW even polled yet” takes its place.

```
bsw_forsa <- polls |> filter(Pollster == "Forsa", !is.na(BSW))
bsw_forsa
```

➤ OUTPUT

```
# A tibble: 121 × 12
  Poll_ID Pollster Date      n Period CDUCSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT02240123K Forsa 2024-01-23 2503 16.01... 31 14 14 4 4
2 BT02240130L Forsa 2024-01-30 2506 23.01... 32 15 14 3 3
3 BT02240206M Forsa 2024-02-06 2503 30.01... 31 15 14 4 3
4 BT02240213N Forsa 2024-02-13 2502 06.02... 31 15 14 4 3
5 BT02240220O Forsa 2024-02-20 2504 13.02... 30 15 14 5 3
6 BT02240227P Forsa 2024-02-27 2503 20.02... 30 14 15 5 3
7 BT02240305Q Forsa 2024-03-05 2506 27.02... 31 15 14 5 3
8 BT02240312R Forsa 2024-03-12 2502 05.03... 30 15 14 5 3
9 BT02240319S Forsa 2024-03-19 2500 12.03... 31 16 13 4 3
10 BT02240326T Forsa 2024-03-26 2503 19.03... 31 16 13 4 3
# i 111 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

`filter()`'s comma is just a logical AND — spelled out with `&` gives the identical result.

```
bsw_forsa_and <- polls |> filter(Pollster == "Forsa" & !is.na(BSW))
identical(bsw_forsa, bsw_forsa_and)
```

➤ OUTPUT

```
[1] TRUE
```

`|` is OR: true if either side is. `&` is AND: true only if both sides are — the same two conditions, either combination.

```
polls |> filter(AfD > 25 | CDUCSU > 35)
```

➤ OUTPUT

```
# A tibble: 377 × 12
  Poll_ID Pollster Date n Period CDUCSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT03170128O Verian 2017-01-28 1996 19.01... 37 23 10 6 10
10 BT02170404C Forsa 2017-04-04 2504 27.03... 36 29 7 5 9
# i 367 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls |> filter(AfD > 25 & CDUCSU > 35)
```

➤ OUTPUT

```
# A tibble: 0 × 12
# i 12 variables: Poll_ID <chr>, Pollster <chr>, Date <date>, n <dbl>,
# Period <chr>, CDUCSU <dbl>, SPD <dbl>, Gruene <dbl>, FDP <dbl>,
# Linke <dbl>, AfD <dbl>, BSW <dbl>
```

Conditions chain as far as needed, but mixing `&` and `|` without parentheses is a classic trap: `&` binds tighter than `|`, so this reads as “(Forsa **and** recent) **or** any high-AfD poll” — not “Forsa **and** (recent **or** high-AfD)”.

```
polls |> filter(Pollster == "Forsa" & Date >= as.Date("2026-01-01") | AfD > 20)
```

➤ OUTPUT

```
# A tibble: 477 × 12
  Poll_ID Pollster Date n Period CDUCSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT05230626M INSA 2023-06-26 2004 23.06.... 26.5 19.5 13.5 6.5 4.5
2 BT052307030 INSA 2023-07-03 2897 30.06.... 25.5 19 14.5 6.5 5
3 BT13230707F YouGov 2023-07-07 1694 30.06.... 27 18 14 6 6
4 BT05230710Q INSA 2023-07-10 2008 07.07.... 26 19 14 7 5
5 BT05230717S INSA 2023-07-17 2004 14.07.... 27 18 14 7 5
6 BT05230722T INSA 2023-07-22 1266 17.07.... 26 18 14 7 5
7 BT05230725U INSA 2023-07-25 2006 21.07.... 26.5 18.5 14 7.5 4.5
8 BT05230729V INSA 2023-07-29 1200 24.07.... 26 18 14 7 5
9 BT05230731W INSA 2023-07-31 2003 28.07.... 26.5 18.5 14 6.5 5
10 BT02230801L Forsa 2023-08-01 2500 25.07.... 25 17 15 6 5
# i 467 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

Parentheses make the intended grouping explicit.

```
polls |> filter(Pollster == "Forsa" & (Date >= as.Date("2026-01-01") | AfD > 20))
```

➤ OUTPUT

```
# A tibble: 94 × 12
  Poll_ID Pollster Date      n Period CDUCSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT02230801L Forsa 2023-08-01 2500 25.07... 25 17 15 6 5
2 BT02230808M Forsa 2023-08-08 2502 01.08... 25 17 15 7 4
3 BT02230815N Forsa 2023-08-15 2501 08.08... 25 17 15 7 4
4 BT02230829P Forsa 2023-08-29 2504 22.08... 26 17 14 7 4
5 BT02230905Q Forsa 2023-09-05 2506 29.08... 27 16 14 7 4
6 BT02230912R Forsa 2023-09-12 2505 05.09... 27 17 13 7 4
7 BT02230919S Forsa 2023-09-19 2504 12.09... 27 17 14 6 4
8 BT02230926T Forsa 2023-09-26 2503 19.09... 27 17 14 6 4
9 BT02231017W Forsa 2023-10-17 2501 10.10... 32 14 14 4 4
10 BT02231024X Forsa 2023-10-24 2504 17.10... 31 14 14 5 4
# i 84 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

`ifelse()` gives a vectorised yes/no across a whole column, no function needed for something this simple.

```
polls |> mutate(threshold = ifelse(FDP >= 5, "cleared threshold", "missed threshold"))
```

➤ OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date      n Period CDUCSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, threshold <chr>
```

Writing your own functions

A function: a name, arguments in `()`, a curly-brace body, and its last line — the one with nothing assigned to it — is what gets returned.

```
custom_year <- function(date_var) {
  year_num <- date_var |> year()
  year_chr <- year_num |> as.character()
  year_chr
}
```

The exact same logic, piped straight through — no need to name every intermediate step.

```
custom_year <- function(date_var) {
  date_var |> year() |> as.character()
}
```

Calling it directly, and piping a date into it, both work the same way.

```
as.Date("2026-09-03")
```

> OUTPUT

```
[1] "2026-09-03"
```

```
Sys.Date()
```

> OUTPUT

```
[1] "2026-09-03"
```

```
Sys.Date() |> year()
```

> OUTPUT

```
[1] 2026
```

```
Sys.Date() |> custom_year()
```

> OUTPUT

```
[1] "2026"
```

```
custom_year(Sys.Date())
```

> OUTPUT

```
[1] "2026"
```

Used inside `mutate()` like any other function — `year()` alone gives a number, `custom_year()` a character string.

```
polls |> mutate(year = year(Date))
```

➤ OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date n Period CDU/CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, year <dbl>
```

```
polls |> mutate(year = custom_year(Date))
```

➤ OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date n Period CDU/CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, year <chr>
```

Default arguments are used unless the caller overrides them.

```
custom_year_correction <- function(date_var, lag = 1) {
  actual_year <- date_var |> year()
  actual_year + lag
}
```

```
Sys.Date() |> custom_year_correction()
```

➤ OUTPUT

```
[1] 2027
```

```
Sys.Date() |> custom_year_correction(2)
```

> OUTPUT

```
[1] 2028
```

```
polls |> mutate(year = custom_year_correction(Date, 3))
```

> OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date      n Period CDU/CSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa    2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian    2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa    2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian    2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa    2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian    2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa    2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian    2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, year <dbl>
```

Anonymous functions

A one-off function with no name — three spellings of the same idea. `~ .x` already showed up on Day 3 (`across()` + `replace_values()`); this is where it's properly introduced, alongside its base R equivalents.

```
(function(x) x / 100)(37) # spelled out
```

> OUTPUT

```
[1] 0.37
```

```
(\(x) x / 100)(37) # base R shorthand (R >= 4.1)
```

> OUTPUT

```
[1] 0.37
```

Piping into an anonymous function needs an extra pair of parentheses around the function itself.


```
37 |> (\(x) x / 100)()
```

>_ OUTPUT

```
[1] 0.37
```

purrr's formula shorthand only works inside purrr calls, never standalone.

```
map_dbl(37, ~ .x / 100)
```

>_ OUTPUT

```
[1] 0.37
```

For loops

for (iterator in sequence) { body } — walking through `party_cols` one entry at a time.

```
for (party in party_cols) {  
  print(str_glue("{party}: Lovely Party!"))  
}
```

>_ OUTPUT

```
CDUCSU: Lovely Party!  
SPD: Lovely Party!  
Gruene: Lovely Party!  
FDP: Lovely Party!  
Linke: Lovely Party!  
AfD: Lovely Party!  
BSW: Lovely Party!
```

A loop can also accumulate a result across iterations — start with an initial value outside the loop, update it each pass.

```
x <- 0
for (i in 1:10) {
  x <- x + i
  print(str_glue("{x}"))
}
```

> OUTPUT

```
1
3
6
10
15
21
28
36
45
55
```

x

> OUTPUT

```
[1] 55
```

For a loop with two iterators, reshape to long format first, then pull the two columns to loop over out as plain vectors with `pluck()` — the same job as `$/[]`, but pipeable.

```
polls_long <- polls |>
  pivot_longer(CDUCSU:BSW, names_to = "Party", values_to = "share")

pollsters <- polls_long |> pluck("Pollster") |> unique()
parties <- polls_long |> pluck("Party") |> unique()
```

A loop inside a loop — every pollster x party pair.

```
for (party in parties) {  
  for (pollster in pollsters) {  
    print(str_glue("Really shocked about today's polling results for {party} by  
{pollster} :o"))  
  }  
}
```

➤ OUTPUT

Really shocked about today's polling results for CDU/CSU by Forsa :o
Really shocked about today's polling results for CDU/CSU by Infratest dimap :o
Really shocked about today's polling results for CDU/CSU by Verian :o
Really shocked about today's polling results for CDU/CSU by YouGov :o
Really shocked about today's polling results for CDU/CSU by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for CDU/CSU by GMS :o
Really shocked about today's polling results for CDU/CSU by Allensbach :o
Really shocked about today's polling results for CDU/CSU by INSA :o
Really shocked about today's polling results for SPD by Forsa :o
Really shocked about today's polling results for SPD by Infratest dimap :o
Really shocked about today's polling results for SPD by Verian :o
Really shocked about today's polling results for SPD by YouGov :o
Really shocked about today's polling results for SPD by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for SPD by GMS :o
Really shocked about today's polling results for SPD by Allensbach :o
Really shocked about today's polling results for SPD by INSA :o
Really shocked about today's polling results for Gruene by Forsa :o
Really shocked about today's polling results for Gruene by Infratest dimap :o
Really shocked about today's polling results for Gruene by Verian :o
Really shocked about today's polling results for Gruene by YouGov :o
Really shocked about today's polling results for Gruene by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for Gruene by GMS :o
Really shocked about today's polling results for Gruene by Allensbach :o
Really shocked about today's polling results for Gruene by INSA :o
Really shocked about today's polling results for FDP by Forsa :o
Really shocked about today's polling results for FDP by Infratest dimap :o
Really shocked about today's polling results for FDP by Verian :o
Really shocked about today's polling results for FDP by YouGov :o
Really shocked about today's polling results for FDP by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for FDP by GMS :o
Really shocked about today's polling results for FDP by Allensbach :o
Really shocked about today's polling results for FDP by INSA :o
Really shocked about today's polling results for Linke by Forsa :o
Really shocked about today's polling results for Linke by Infratest dimap :o
Really shocked about today's polling results for Linke by Verian :o
Really shocked about today's polling results for Linke by YouGov :o
Really shocked about today's polling results for Linke by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for Linke by GMS :o
Really shocked about today's polling results for Linke by Allensbach :o
Really shocked about today's polling results for Linke by INSA :o
Really shocked about today's polling results for AfD by Forsa :o
Really shocked about today's polling results for AfD by Infratest dimap :o
Really shocked about today's polling results for AfD by Verian :o
Really shocked about today's polling results for AfD by YouGov :o
Really shocked about today's polling results for AfD by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for AfD by GMS :o
Really shocked about today's polling results for AfD by Allensbach :o
Really shocked about today's polling results for AfD by INSA :o
Really shocked about today's polling results for BSW by Forsa :o
Really shocked about today's polling results for BSW by Infratest dimap :o
Really shocked about today's polling results for BSW by Verian :o
Really shocked about today's polling results for BSW by YouGov :o
Really shocked about today's polling results for BSW by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for BSW by GMS :o

```
Really shocked about today's polling results for BSW by Allensbach :o
Really shocked about today's polling results for BSW by INSA :o
```

expand_grid(): the tidyverse alternative to nested loops

The exact same pollster x party combinations as above, built without nesting — `expand_grid()` creates every pairing as a tibble, one row per combination.

```
grid <- expand_grid(party = parties, pollster = pollsters)
grid
```

> OUTPUT

```
# A tibble: 56 × 2
  party pollster
<chr> <chr>
1 CDU/CSU Forsa
2 CDU/CSU Infratest dimap
3 CDU/CSU Verian
4 CDU/CSU YouGov
5 CDU/CSU Forschungsgruppe Wahlen
6 CDU/CSU GMS
7 CDU/CSU Allensbach
8 CDU/CSU INSA
9 SPD Forsa
10 SPD Infratest dimap
# i 46 more rows
```

The same message from the nested loop, built once for every row via `mutate()` — no loop needed, `str_glue()` is vectorised.

```
grid |> mutate(output = str_glue("Really shocked about today's polling results for
{party} by {pollster} :o"))
```

> OUTPUT

```
# A tibble: 56 × 3
  party pollster output
<chr> <chr> <glue>
1 CDU/CSU Forsa Really shocked about today's polling results ...
2 CDU/CSU Infratest dimap Really shocked about today's polling results ...
3 CDU/CSU Verian Really shocked about today's polling results ...
4 CDU/CSU YouGov Really shocked about today's polling results ...
5 CDU/CSU Forschungsgruppe Wahlen Really shocked about today's polling results ...
6 CDU/CSU GMS Really shocked about today's polling results ...
7 CDU/CSU Allensbach Really shocked about today's polling results ...
8 CDU/CSU INSA Really shocked about today's polling results ...
9 SPD Forsa Really shocked about today's polling results ...
10 SPD Infratest dimap Really shocked about today's polling results ...
# i 46 more rows
```

`pluck()` again to get that column out as a plain character vector...

```
grid |>
  mutate(output = str_glue("Really shocked about today's polling results for {party} by
{pollster} :o")) |>
  pluck("output")
```

➤ OUTPUT

Really shocked about today's polling results for CDU/CSU by Forsa :o
Really shocked about today's polling results for CDU/CSU by Infratest dimap :o
Really shocked about today's polling results for CDU/CSU by Verian :o
Really shocked about today's polling results for CDU/CSU by YouGov :o
Really shocked about today's polling results for CDU/CSU by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for CDU/CSU by GMS :o
Really shocked about today's polling results for CDU/CSU by Allensbach :o
Really shocked about today's polling results for CDU/CSU by INSA :o
Really shocked about today's polling results for SPD by Forsa :o
Really shocked about today's polling results for SPD by Infratest dimap :o
Really shocked about today's polling results for SPD by Verian :o
Really shocked about today's polling results for SPD by YouGov :o
Really shocked about today's polling results for SPD by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for SPD by GMS :o
Really shocked about today's polling results for SPD by Allensbach :o
Really shocked about today's polling results for SPD by INSA :o
Really shocked about today's polling results for Gruene by Forsa :o
Really shocked about today's polling results for Gruene by Infratest dimap :o
Really shocked about today's polling results for Gruene by Verian :o
Really shocked about today's polling results for Gruene by YouGov :o
Really shocked about today's polling results for Gruene by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for Gruene by GMS :o
Really shocked about today's polling results for Gruene by Allensbach :o
Really shocked about today's polling results for Gruene by INSA :o
Really shocked about today's polling results for FDP by Forsa :o
Really shocked about today's polling results for FDP by Infratest dimap :o
Really shocked about today's polling results for FDP by Verian :o
Really shocked about today's polling results for FDP by YouGov :o
Really shocked about today's polling results for FDP by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for FDP by GMS :o
Really shocked about today's polling results for FDP by Allensbach :o
Really shocked about today's polling results for FDP by INSA :o
Really shocked about today's polling results for Linke by Forsa :o
Really shocked about today's polling results for Linke by Infratest dimap :o
Really shocked about today's polling results for Linke by Verian :o
Really shocked about today's polling results for Linke by YouGov :o
Really shocked about today's polling results for Linke by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for Linke by GMS :o
Really shocked about today's polling results for Linke by Allensbach :o
Really shocked about today's polling results for Linke by INSA :o
Really shocked about today's polling results for AfD by Forsa :o
Really shocked about today's polling results for AfD by Infratest dimap :o
Really shocked about today's polling results for AfD by Verian :o
Really shocked about today's polling results for AfD by YouGov :o
Really shocked about today's polling results for AfD by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for AfD by GMS :o
Really shocked about today's polling results for AfD by Allensbach :o
Really shocked about today's polling results for AfD by INSA :o
Really shocked about today's polling results for BSW by Forsa :o
Really shocked about today's polling results for BSW by Infratest dimap :o
Really shocked about today's polling results for BSW by Verian :o
Really shocked about today's polling results for BSW by YouGov :o
Really shocked about today's polling results for BSW by Forschungsgruppe Wahlen :o
Really shocked about today's polling results for BSW by GMS :o

```
Really shocked about today's polling results for BSW by Allensbach :o  
Really shocked about today's polling results for BSW by INSA :o
```

...and `str_c(collapse = ...)` to fold the whole vector down into one single string.


```
grid |>
  mutate(output = str_glue("Really shocked about today's polling results for {party} by
{pollster} :o")) |>
  pluck("output") |> str_c(collapse = "; ")
```

> OUTPUT

```
[1] "Really shocked about today's polling results for CDU/CSU by Forsa :o; Really
shocked about today's polling results for CDU/CSU by Infratest dimap :o; Really
shocked about today's polling results for CDU/CSU by Verian :o; Really shocked about
today's polling results for CDU/CSU by YouGov :o; Really shocked about today's polling
results for CDU/CSU by Forschungsgruppe Wahlen :o; Really shocked about today's
polling results for CDU/CSU by GMS :o; Really shocked about today's polling results
for CDU/CSU by Allensbach :o; Really shocked about today's polling results for CDU/CSU
by INSA :o; Really shocked about today's polling results for SPD by Forsa :o;
Really shocked about today's polling results for SPD by Infratest dimap :o; Really
shocked about today's polling results for SPD by Verian :o; Really shocked about
today's polling results for SPD by YouGov :o; Really shocked about today's polling
results for SPD by Forschungsgruppe Wahlen :o; Really shocked about today's polling
results for SPD by GMS :o; Really shocked about today's polling results for SPD
by Allensbach :o; Really shocked about today's polling results for SPD by INSA :o;
Really shocked about today's polling results for Gruene by Forsa :o; Really shocked
about today's polling results for Gruene by Infratest dimap :o; Really shocked
about today's polling results for Gruene by Verian :o; Really shocked about today's
polling results for Gruene by YouGov :o; Really shocked about today's polling results
for Gruene by Forschungsgruppe Wahlen :o; Really shocked about today's polling
results for Gruene by GMS :o; Really shocked about today's polling results for
Gruene by Allensbach :o; Really shocked about today's polling results for Gruene
by INSA :o; Really shocked about today's polling results for FDP by Forsa :o;
Really shocked about today's polling results for FDP by Infratest dimap :o; Really
shocked about today's polling results for FDP by Verian :o; Really shocked about
today's polling results for FDP by YouGov :o; Really shocked about today's polling
results for FDP by Forschungsgruppe Wahlen :o; Really shocked about today's polling
results for FDP by GMS :o; Really shocked about today's polling results for FDP
by Allensbach :o; Really shocked about today's polling results for FDP by INSA :o;
Really shocked about today's polling results for Linke by Forsa :o; Really shocked
about today's polling results for Linke by Infratest dimap :o; Really shocked about
today's polling results for Linke by Verian :o; Really shocked about today's polling
results for Linke by YouGov :o; Really shocked about today's polling results for
Linke by Forschungsgruppe Wahlen :o; Really shocked about today's polling results
for Linke by GMS :o; Really shocked about today's polling results for Linke by
Allensbach :o; Really shocked about today's polling results for Linke by INSA :o;
Really shocked about today's polling results for AfD by Forsa :o; Really shocked
about today's polling results for AfD by Infratest dimap :o; Really shocked about
today's polling results for AfD by Verian :o; Really shocked about today's polling
results for AfD by YouGov :o; Really shocked about today's polling results for AfD
by Forschungsgruppe Wahlen :o; Really shocked about today's polling results for AfD
by GMS :o; Really shocked about today's polling results for AfD by Allensbach :o;
Really shocked about today's polling results for AfD by INSA :o; Really shocked
about today's polling results for BSW by Forsa :o; Really shocked about today's
polling results for BSW by Infratest dimap :o; Really shocked about today's polling
results for BSW by Verian :o; Really shocked about today's polling results for BSW
by YouGov :o; Really shocked about today's polling results for BSW by Forschungsgruppe
Wahlen :o; Really shocked about today's polling results for BSW by GMS :o; Really
shocked about today's polling results for BSW by Allensbach :o; Really shocked about
today's polling results for BSW by INSA :o"
```

Rowwise

A function written for two scalars works fine called on its own...

```
weakest_party_result <- function(party1, party2) {  
  min(party1, party2)  
}  
  
weakest_party_result(13, 14)
```

> OUTPUT

```
[1] 13
```

...but breaks silently inside `mutate()`: `min()` sees the *whole* FDP and Linke columns at once, not one row's pair of values, so every row gets the same overall minimum.

```
polls |> head(100) |>  
  mutate(weakest = weakest_party_result(FDP, Linke))
```

> OUTPUT

```
# A tibble: 100 × 13  
  Poll_ID Pollster Date      n Period CDU/CSU SPD Gruene FDP Linke  
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>  
1 BT021701040 Forsa    2017-01-04 2001 27.12... 37 21 10 6 9  
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9  
3 BT03170107L Verian    2017-01-07 834 03.01... 38 22 10 5 9  
4 BT02170111P Forsa    2017-01-11 2501 02.01... 37 20 10 6 9  
5 BT03170114M Verian    2017-01-14 1878 06.01... 37 21 10 6 10  
6 BT02170118Q Forsa    2017-01-18 2503 09.01... 38 21 9 6 9  
7 BT03170121N Verian    2017-01-21 2805 12.01... 36 21 9 6 11  
8 BT02170125R Forsa    2017-01-25 2504 16.01... 37 21 10 6 9  
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8  
10 BT031701280 Verian    2017-01-28 1996 19.01... 37 23 10 6 10  
# i 90 more rows  
# i 3 more variables: AfD <dbl>, BSW <dbl>, weakest <dbl>
```

`rowwise()` fixes it — the function is applied one row at a time instead.

```
polls |> head(100) |>
  rowwise() |>
  mutate(weakest = weakest_party_result(FDP, Linke))
```

> OUTPUT

```
# A tibble: 100 × 13
# Rowwise:
  Poll_ID      Pollster Date      n Period CDU/CSU SPD Gruene FDP Linke
  <chr>      <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa    2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian    2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa    2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian    2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa    2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian    2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa    2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian    2017-01-28 1996 19.01... 37 23 10 6 10
# i 90 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, weakest <dbl>
```

Try it: who cleared the threshold?

Your task: write a function that reports whether a party's poll share cleared a threshold — default the 5% electoral threshold. Use it to flag every party's status in the full `polls` table via `mutate()`, then call it directly on CDU/CSU's latest share against the 33.3% blocking-minority mark instead.

```
clears_threshold <- function(share, threshold = 5) {
  ifelse(share >= threshold, "cleared threshold", "missed threshold")
}

latest_poll <- polls |> slice_max(Date, n = 1, with_ties = FALSE)

polls |> mutate(fdp_status = clears_threshold(FDP))
```

➤ OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date n Period CDU/CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, fdp_status <chr>
```

```
clears_threshold(latest_poll$CDU/CSU, threshold = 33.3) # blocking-minority mark
```

➤ OUTPUT

```
[1] "missed threshold"
```

Try it: counting who cleared

Your task: using the latest poll, report every party's threshold status one at a time with a `for` loop, and count how many parties cleared it with an accumulator.

```
n_cleared <- 0
for (party in party_cols) {
  status <- clears_threshold(latest_poll[[party]])
  print(str_glue("{party}: {status}"))
  if (!is.na(status) && status == "cleared threshold") # BSW: NA where a pollster hasn't
    tracked it yet
    n_cleared <- n_cleared + 1
}
```

> OUTPUT

```
CDUCSU: cleared threshold
SPD: cleared threshold
Gruene: cleared threshold
FDP: missed threshold
Linke: cleared threshold
AfD: cleared threshold
BSW: NA
```

n_cleared

> OUTPUT

```
[1] 5
```

Try it: average share by pollster

Your task: compute each pollster's average share for every party, one pollster-party combination at a time, with a nested `for` loop.

```

pollster_party_mean <- function(pollster_name, party_name) {
  polls_long |>
    filter(Pollster == pollster_name, Party == party_name) |>
    pull(share) |>
    mean(na.rm = TRUE) |>
    round(1)
}

for (pollster in pollsters) {
  for (party in parties) {
    avg <- pollster_party_mean(pollster, party)
    print(str_glue("{pollster} / {party}: {avg}%"))
  }
}

```

➤ OUTPUT

Forsa / CDU/CSU: 29.6%
Forsa / SPD: 17.2%
Forsa / Grüne: 15.8%
Forsa / FDP: 7%
Forsa / Linke: 7.4%
Forsa / AfD: 14.4%
Forsa / BSW: 4.3%
Infratest dimap / CDU/CSU: 29.9%
Infratest dimap / SPD: 17.7%
Infratest dimap / Grüne: 16.3%
Infratest dimap / FDP: 7.3%
Infratest dimap / Linke: 6.9%
Infratest dimap / AfD: 14.6%
Infratest dimap / BSW: 4.8%
Verian / CDU/CSU: 29.5%
Verian / SPD: 18.7%
Verian / Grüne: 16.3%
Verian / FDP: 7.7%
Verian / Linke: 7.5%
Verian / AfD: 13.4%
Verian / BSW: 4.4%
YouGov / CDU/CSU: 29.6%
YouGov / SPD: 17.7%
YouGov / Grüne: 14.5%
YouGov / FDP: 6.8%
YouGov / Linke: 7.9%
YouGov / AfD: 15.9%
YouGov / BSW: 5.6%
Forschungsgruppe Wahlen / CDU/CSU: 29%
Forschungsgruppe Wahlen / SPD: 17.2%
Forschungsgruppe Wahlen / Grüne: 17%
Forschungsgruppe Wahlen / FDP: 6.5%
Forschungsgruppe Wahlen / Linke: 7.2%
Forschungsgruppe Wahlen / AfD: 15.5%
Forschungsgruppe Wahlen / BSW: 4.8%
GMS / CDU/CSU: 29.5%
GMS / SPD: 16.8%
GMS / Grüne: 16.2%
GMS / FDP: 7.4%
GMS / Linke: 6.9%
GMS / AfD: 15.4%
GMS / BSW: 4.3%
Allensbach / CDU/CSU: 30.4%
Allensbach / SPD: 17.9%
Allensbach / Grüne: 15.9%
Allensbach / FDP: 7.2%
Allensbach / Linke: 7.1%
Allensbach / AfD: 14.9%
Allensbach / BSW: 5.6%
INSA / CDU/CSU: 27.1%
INSA / SPD: 17.7%
INSA / Grüne: 13.8%
INSA / FDP: 6.2%
INSA / Linke: 6.1%

INSA / AfD: 19.2%
INSA / BSW: 5.6%